````
---
title: "lab_11"
author: "Derek Willis"
date: "2025-01-16"
output: html_document
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

## You will need

-   Our usual libraries for working with data, plus rvest and lubridate.

## Load libraries and establish settings

**Task** Create a codeblock and load appropriate packages and settings for this lab.

```{r}
library(rvest)
library(lubridate)
library(tidyverse)
library(janitor)
library(tidytext)
```


Let's get to scraping.

## Questions

**Q1**. Scrape the listing of available Maryland physician disciplinary alerts at <https://www.mbp.state.md.us/disciplinary_2024.aspx> into a dataframe. You should have three columns, one of which is a date, so make sure the date column has a date datatype. What's the most common sanction, and how many times is it mentioned in the 2024 alerts?

**A1** Summary suspension is the most common sanction. The "affirmed" kind is the highest specific term tied with surrender of license at 7, but "summary suspension" at 6 and "permanent surrender of license" at 4 aren't far off, and after that the table tapers off into niche sanctions of different sentences and of only 1 count each. Therefore, summary suspension is the highest at 13 plus a number of more niche kinds versus 11 surrenders of license.

```{r}
alerts <- read_html("https://www.mbp.state.md.us/disciplinary_2024.aspx") |>
  html_elements("table") |>
  html_table()

alerts_table <- alerts[[1]] |>
  clean_names() |>
  mutate(date = mdy(date))

alerts_table |>
  count(sanction, sort = TRUE)
```

**Q2** Next, let's scrape the list of press releases from Maryland's Office of the Public Defender, <https://www.opd.state.md.us/press-releases>. This isn't a table, so you'll need to use `html_elements()` and your browser's inspector and do some clean up on the results. The result should be a dataframe with two columns that contain the date and title, and the date column should have a date datatype. The challenge here is figuring out how to isolate the releases.
````

When you finish scraping into a dataframe, write code to find the press releases that have the word "police" in the title.

A2 9 press releases with the term "police".

```{r}
press_releases <- read_html("https://www.opd.state.md.us/press-releases") |>
  html_elements("div[data-testid='richTextElement'] a") |>
  html_text() |>
  tibble(text = _) |>
  mutate(text = str_squish(text)) |>
  filter(str_detect(text, ":")) |>
  separate(text, into = c("date", "title"), sep = ": ", extra = "merge") |>
  mutate(
    date = mdy(date),
    title = str_trim(title)
  )

police_releases <- press_releases |>
  filter(str_detect(title, fixed("police", ignore_case = TRUE)))
```

Q3 Sen. Chris Van Hollen, D-Maryland, has posted press releases at <https://www.vanhollen.senate.gov/news/press-releases>. It would be great to have them in a dataframe that has the following columns: date, title and url.

To do this, you will need to scrape the page's html and save that to a variable, and *then* extract the dates, titles and urls into separate dataframes using html_elements(). There are two different elements that contain the press release URL; pick one of them, and you may need to invoke an HTML `class` value to do it. The function `html_text()` pulls out the contents of a tag, but for urls we want the HTML attribute. Rvest gives you a way to extract the URL from a link; google to find out what it is. Remember how we turn a list into a dataframe.

At the end, you'll have three dataframes that you want to combine into a single dataframe. When we want to combine the rows of identical dataframes, we used `bind_rows()`. If you were combining columns instead of rows, there's a similar function. Find out what it is and use it to put all of the dataframes together into a single one.

When you're done, rename the columns so they make sense, then make sure the date column is an actual date.

Finally, tell me what questions you could ask of this data if you had all of his press releases. Be creative.

A3You could find how often terms related to hot news or subjects are mentioned like "Trump" or "Venezuela". Or collaborating senators like "Alsobrooks". There isn't much else I can think of to look for besides filtering the title column since all the urls are for releases for Van Hollen's website and the releases are all just gradually sent out over the past few weeks.

```{r}
vanhollen <- read_html("https://www.vanhollen.senate.gov/news/press-releases")

date <- vanhollen |>
  html_elements(".Heading--time") |>
  html_text() |>
  tibble(date = _)

title <- vanhollen |>
  html_elements(".ArticleTitle") |>
  html_text() |>
  tibble(title = _)
```

```
url <- vanhollen |>
  html_elements(".ArticleBlock__titleContainer a") |>
  html_attr("href") |>
  tibble(url = _)

vanhollen_table <- bind_cols(date, title, url) |>
  mutate(
    date = mdy(date),
    title = str_trim(title)
  )
```